LLM-Based Knowledge Graph Construction and Retrieval for Legal Document Analysis

Samuel Bagín, Marek Vančo
Faculty of Electrical Engineering and Information Technology
Slovak University of Technology
Bratislava, Slovakia
xbagins@stuba.sk, marek_vanco@stuba.sk

***Abstract*—Legal documents combine long, deeply structured text with numeric tables and references to paragraphs, which makes them difficult to process using classical retrieval-augmented generation. This work presents a pipeline that transforms Slovak financial law into a navigable knowledge graph and answers natural language questions over it. The philosophy of the paper is that legal reasoning rarely depends on a single passage. Instead, it follows chains of concepts such as law, paragraph, obligation and calculation mechanism. A knowledge graph makes these chains explicit and allows an agent to walk them step by step. We combine layout-aware preprocessing, an ontology-guided extraction stage inspired by ODKE+, and a state-machine retrieval stage inspired by ARK-V1, all orchestrated inside a three-stage KG-GPT pipeline. The result is a system that grounds every answer in explicit graph evidence.**

***Index Terms*—knowledge graph, large language models, legal document analysis, GraphRAG, Neo4j, information extraction**

I. Introduction

With the growing use of Large Language Models (LLMs) across various domains, there is an increasing need for effective methods to analyze and process specific document types. Slovak legal and financial documents are long, heavily structured and full of cross-references. A typical act contains nested paragraphs, numbered items, tables and formulas that describe how to compute a tax. A lawyer or ordinary person who wants to answer a question, such as "which paragraph defines a given obligation" or "how a tax base is calculated", has to follow a chain of references across several pages. Classical retrieval-augmented generation (RAG) over plain text performs poorly in this setting. A single chunk rarely contains the full answer, and the model loses the structural information that paragraphs and tables carry. LLMs alone often struggle with the complex structures and relationships in legal documents, leading to hallucinations and limitations in providing precise information [9].

This paper describes a complete pipeline that turns such documents into a knowledge graph (KG) stored in Neo4j [10] graph database and then queries the graph with a large language model (LLM) agent. The construction phase is inspired by ODKE+ [15], which uses an ontology to guide open-domain extraction. The query phase uses the three-stage KG-GPT framework [13] and plugs the ARK-V1 state-machine retrieval agent [14] into its middle stage. The orchestration layer is built on top of LangChain [11] and uses several LLMs for different tasks, such as GPT-5.4-mini, Gemini 2.5 Flash and Pro models.

The main contributions are a preprocessing stage that detects tables and formulas with a document layout detector, a three-step extraction stage that produces a stable legal ontology, and a retrieval stage that walks the graph in small verifiable steps instead of generating a single complex Cypher query. The rest of the paper follows: Section II reviews related work, Section III describes document processing, Section IV describes search and retrieval, and Section V conclusion.

II. State of the Art

Building knowledge graphs from text has traditionally relied on pipelines of specialized NER and relation extraction modules, but recent work increasingly turns to LLMs to carry out the entire extraction end-to-end. The GraphRAG approach of Edge et al. [2] demonstrated that a knowledge graph built by an LLM, combined with community summarization, outperforms classical RAG on query-focused summarization. However, their graphs are extracted in an open-domain fashion and the schema is not explicitly controlled, which is a problem for legal text where the ontology must be stable and auditable. ODKE+ [15] addresses this by combining open-domain detection with an ontology and a schema-driven extraction step. To improve fidelity, KnoBuilder [3] utilizes an iterative agentic loop featuring planning and self-refining modules, while GraphJudge [4] employs a discriminative judge to reach 90% triple accuracy. On the retrieval side, KG-GPT [13] proposes a three-stage framework of segmentation, retrieval, and inference. ARK-V1 [14] fills the gap with a state-machine agent that repeatedly selects relations and advances anchors to neighbors, ensuring every reasoning hop is explicit and auditable. This mirrors the discrete decision-making in Reasoning with Trees (RwT) [5], which utilizes Monte Carlo Tree Search—balancing exploration and exploitation via the UCB formula—to navigate complex paths. For dynamic data, GMeLLo [6] translates natural language into SPARQL, while LightRAG [7] provides cost-efficient dual-level search. Additionally, LEGO-GraphRAG [8] improves reasoning by decomposing retrieval into specialized path refinement modules. LLM security and privacy considerations studied in Security and Privacy in Large Language Models [1] also matter in the legal domain, where hallucinated citations can have practical

consequences. Grounding every answer in explicit graph evidence is one way to mitigate this risk, and it is one of the main motivations for the architecture proposed here.

III. Document Processing

Document processing transforms a raw PDF of a Slovak law into a populated knowledge graph. It has three substages: preprocessing, extraction and database insertion. Fig. 1 illustrates the process pipeline.

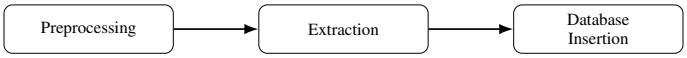

Fig. 1: Process pipeline.

A. Preprocessing

Raw PDF text extraction is not enough for legal documents. Tables, for instance, span across several pages and carry hierarchical information (section, chapter, item) that is destroyed if the table is read as free text. Isolated formulas are also frequent and should be kept as image regions rather than fed into a text splitter as garbled characters.

The preprocessing stage converts each page of the PDF into and image at 200 DPI and applies a document layout detector based on the DocLayout-YOLO model. The detector is configured to recognise five target classes: *table picture figure isolate_formula* and *formula_caption*. Detections with confidence below $\tau = 0.3$ are discarded. For every retained detection the bounding box is cropped from the page image and saved as a PNG file. Consecutive table detections on neighbouring pages are grouped together so that multi-page tables are kept as a single logical unit.

Each table group is the processed in two steps. First a vision-capable Gemini model receives all cropped images of the group together with a system prompt that instructs it to output a single HTML table. The prompt forbids summarization and requires the model to preserve every cell verbatim, and to reuse rowspan and colspan for merged cells. Second, the resulting HTML is fed to another LLM call that transforms it into a hierarchical set of nodes and relationships: a root node for the whole table, parent nodes for each section indicated by rowspan in the first column, and leaf nodes for the individual tariff items. This is the only way to preserve the semantics of a tariff schedule inside a knowledge graph. Interior pages of a multi-page table are excluded from later text extraction to avoid duplicating the same information.

B. Extraction

After preprocessing, the remaining text pages are chunked and passed through a three-step extraction pipeline inspired by ODKE+ [15].

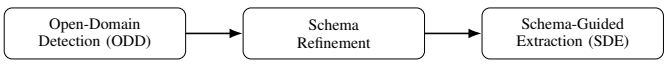

Fig. 2: Extraction pipeline inspired by ODKE+.

1) Open-Domain Detection (ODD): The goal of the first step is to discover, from the document, which entity types and relation types are meaningful. Text is split using a recursive characters splitter with a chunk size of 1200 characters and an overlap of 200 characters. The relatively larger window is used deliberately: at this stage the GPT-5.4-mini model is asked about types, not instances, and a wider context helps it see more than one sentence at a time, which is important in legal language, where a concept such as *tax base* is often introduced several sentences before it is first used. Each chunk c_i is sent in parallel to an agent whose system prompt instructs it to return generic but distinguishable node labels and relationship labels, with an explicit focus on laws, paragraphs, obligations, rights and temporal entities. All Slovak diacritics are normalised to ASCII so that the graph uses a single consistent spelling. The output of this step is a list of per-chunk schemas $S_i = (\mathcal{T}_V^{(i)}, \mathcal{T}_E^{(i)})$. For next step, the per-chunk schemas are merged into a single united set of node types and relation types:$S_{\text{ODD}} = \bigcup_{i=1}^{|C|} S_i$

2) Schema Refinement (SR): The merged schema from ODD is sent to a schema refinemnt using Gemini 2.5 Pro call together with any existing ontology already stored in graph: $S^* = \Phi(S_{\text{ODD}}, S_0)$. The refinement step has a clear linguistic role: it resolves duplicates caused by diacritics or minor spelling variants (for example Dan vs Daň), and semantically simillar types (for example Doklad, DanovyDoklad and HodnovernyDoklad into Doklad). It enforces upper-snake-case for relation types and it refuses to merge semantically distinct categories such as *document* and *person*. A merge log is kept that maps every canonical type back to the raw variants it absorbed, which is useful both for debugging and for auditing. A special type *Paragraph* is always added to the node list, because every legal fact in the graph should be traceableback to the paragraph that states it.

3) Schema-Driven Extraction (SDE): The final extraction step uses the refined S^* schema as a hard constraint. Text is rechunked with a smaller window of 1024 characters and an overlap of 128 characters. This smaller chunk is motivated by a different trade-off: at this stage the GPT-5.4-mini model must extract concrete instances and a shorter window pushes it to produce tighter, better-grounded triples (e_h, r, e_t), where e_h is head entity, e_t is tail entity and r is relationship between them. It reduces the chance that two unrelated facts are blended into one relation, and makes the per-chunk LLM cost predictable. The chunks are sent in parallel, with a bounded concurrency and an automatic pause-and-retry mechanism in case of provider errors (when user hits Tokens Per Minute limit). For each chunk c_i the model returns nodes and relationships, that conform to the refined S^* schema and yields a GraphDocument $G_i = (V_i, E_i, src_i)$. A lightweight post-processing step normalises node IDs to title case, normalises relation names to upper-snake-case, and drops entries with missing source or target. Every extracted node is also linked to a *Chunk* node via an IN_CHUNK relation, which later allows retrieval stage to fetch the original text that supports a given triple.

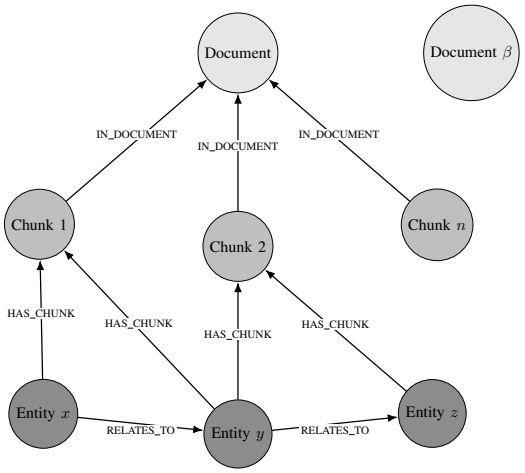

Fig. 3: Knowledge graph structure: document, chunk, entity and relationships. Image taken and edited to our solution from Neo4j [17]

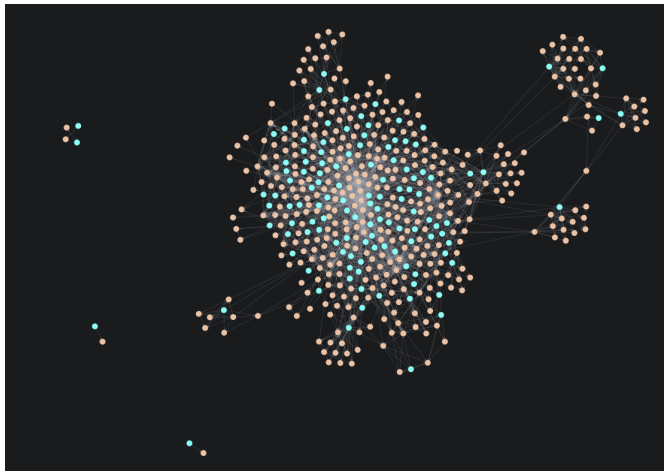

Fig. 4: Graph database in Neo4j Desktop.

C. Database Insertion

Extracted graph documents are inserted into a Neo4j database [10]. Nodes are created with MERGE on their normalised ID so that repeated mentions across chunks collapse into a single entity. Relationships are created in the same MERGE style to avoid duplicate edges. In addition to the extracted nodes and edges, the pipeline also inserts a Document node per source PDF, linked to every Chunk node via an IN_DOCUMENT relation.

IV. Search and Retrieval

The query pipeline follows KG-GPT framework [13]: sentence segmentation, graph retrieval and inference. Fig. 5 summarises the flow.

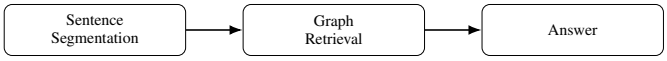

Fig. 5: Query pipeline. Each sub-sentence is retrieved independently.

A. Sentence Segmentation

Legal question are often compound. A single question such as "which paragraph of the law defines the tax base and what is the formula used to compute it" implies two graph facts, not one. The segmentation stage [16] breaks the user question into sub-sentences and for each sub-sentence, extracts up to two anchor entities written in title case. Every sub-sentence is meant to correspond to approximately one graph triple (e_h, r, e_t). The returned object is $(\text{sentence}_j,\ \text{entities}_j)$ If no sub-sentence is produced, the original question is used as a fallback.

B. Graph Retrieval

The middle stage is the most sensitive part of the pipeline, because it is where answers are grounded in actual graph evidence. We implement a variant of the ARK-V1 state machine [14]. For each sub-sentence, and for each anchor entity mentioned in it (up to two), the agent performs a bounded walk of depth at most $K_{max} = 3$.

A single step of the chain has four internal sub-steps. First, the anchor is resolvved: the graph is queried for a node whose ID mathces case-insensitively, or as a fallback contains the raw anchor string. Second, all relation types actually incident to the resolved anchor are enumerated, both outgoing and incoming. This produces the candidate set R^k. Third, the LLM is asked to pick exactly one relation from R^k or to return null. The prompt is rebuilt from scratch at every step and contains only the system instruction, the sub-sentence, the current anchor, the running summary and the candidate list. This keeps context size roughly constat in k. Up to $C_{max} = 2$ retries are allowed, and the choice is validated against R^k on every retry. Fourth, triples of the form (e_{anchor}, rel, u), where u is any entity, are enumerated from the graph, capped at 25 per step, and the LLM selects the relevant indices, summarization the implication of the step and decides whether to continue. If it continues, a tail of one of the selected triples becomes the new anchor and the loop repeats.

Formally, at step k the agent maintains the state $s_k = (a_k, \Sigma_k, E_k)$, where a_k is the current anchor, Σ_k is the running textual summary and E_k is the accumulated set of evidence triples. The transition picks a relation $r_k \in R^k$ and a subset of triples $T_k \subseteq \{(a_k, r_k, t) : t \in V\}$. The update is

$$E_{k+1} = E_k \cup T_k, \qquad a_{k+1} \in \pi_3(T_k) \cup \{a_k\},$$

where π_3 projects a triple onto its tail. The chain terminantes when the agent sets the *continue* flag to false, when R^k is empty or when k reaches K_{max}

Two design choices important noticing. First, the candidate set R^k is always built from the graph itself. This means the agent can never propose a relation that does not exist for the

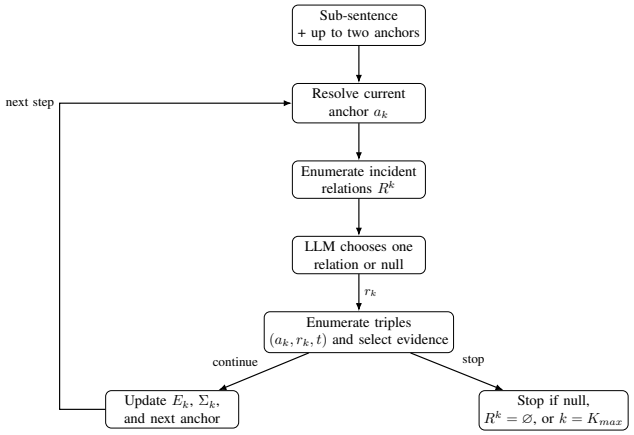

Fig. 6: Simple evidence-retrieval loop over the knowledge graph. For each anchor, the agent resolves the node, enumerates valid relations from the graph, chooses one relation, selects supporting triples, and either continues with a new anchor or stops.

current anchor, which eliminates an entire class of hallucinations. Second, the running summary Σ_k is the only long-term memory carried between steps. The triples themselves are re-enumerated every time. This is what makes the context size predictable even for deep walks. At the end of the per-anchor loop the evidence triples are deduplicated while preserving insertion order.

C. Inference

The final stage aggregates the evidence triples from all sub-sentences into a single list and retrieves the text chunks linked to the involved nodes via the IN_CHUNK relationship. A Gemini 2.5 Flash model is then prompted with three blocks: the original user question, retrieved evidence triples (e_h, r, e_t) as primary evidence, and the retrieved chunks as supplementary context. The prompt instructs the model to prioritise the graph triples and to use the chunks only for wording and context. The output is a short Slovak answer grounded in explicit graph evidence.

V. Conclusion

We presented a GraphRAG pipeline tailored to Slovak financial law. On the construction side, layout-aware preprocessing preserves tables and formulas, and an ODKE+ inspired three-step extraction produces a stable ontology with chunk-level provenance. On the query side, a KG-GPT pipeline with an ARK-V1 state machine in the middle stage walks the graph in small, verifiable hops and grounds every answer in explicit evidence. The two different chunk sizes used in extraction, 1200 characters for open-domain detection and 1024 characters for schema-driven extraction, reflect the different goals of the two stages: wide context for type discovery, narrow context for precise instance extraction. Future work includes a quantitative evaluation on a curated set of Slovak legal questions

Acknowledgment

This paper was supported by DISIC (09I05-03-V2), NEXT (ERASMUS-EDU-2023-CBHE-STRAND-2), EULiST (ERASMUS), CYB-FUT (ERASMUS+), InteRViR (VEGA 1/0605/23).

References

[1] Badhan Chandra Das, M. Hadi Amini, Yanzhao Wu, "Security and Privacy in Large Language Models: A Survey," 2024. https://arxiv.org/pdf/2402.00888

[2] D. Edge et al., "From Local to Global: A GraphRAG Approach to Query-Focused Summarization," *Microsoft Research*, 2024. https://arxiv.org/pdf/2404.16130

[3] Bayron Jossue Serrano Mena, "KnoBuilder: An LLM-Agent for Autonomous and Personalized Knowledge Graph Construction from Unstructured Text", 2025. https://openreview.net/pdf?id=teewCPCv2m

[4] Huang et al., "Can LLMs be Good Graph Judge for Knowledge Graph Construction?," 2025. https://aclanthology.org/2025.emnlp-main.554.pdf

[5] Shen et al., "Reasoning with Trees: MCTS for Multi-Hop Question Answering," 2025. https://aclanthology.org/2025.coling-main.211.pdf

[6] Chent et al., "GMeLLo: LLM-Based Multi-Hop Question Answering with Knowledge Graph Integration in Evolving Environments," 2025. https://arxiv.org/html/2408.15903

[7] Guo et al., "LightRAG: Simple and Fast Retrieval-Augmented Generation," 2024. https://arxiv.org/pdf/2410.05779

[8] Cao et al., "LEGO-GraphRAG: Modularizing Graph-based Retrieval-Augmented Generation for Design Space Exploration," 2025. https://arxiv.org/pdf/2411.05844

[9] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly," *High-Confidence Computing*, vol. 4, no. 2, 2024, Art. no. 100211.

[10] Neo4j, Inc., "Neo4j graph database,". https://neo4j.com/

[11] LangChain, Inc., "LangChain documentation," https://docs.langchain.com/

[12] Slov-Lex, "Legal and information portal," Ministry of Justice of the Slovak Republic. https://www.slov-lex.sk/

[13] Kim, J., Kwon, Y., Jo, Y., and Choi, E. *KG-GPT: A General Framework for Reasoning on Knowledge Graphs Using Large Language Models*. Findings of EMNLP 2023. https://aclanthology.org/2023.findings-emnlp.631.pdf.

[14] Klein, J. F. and Ohnemus, J. *ARK-V1: A State-Machine LLM Agent for Knowledge-Graph Question Answering*. arXiv:2509.18063, 2025. https://arxiv.org/pdf/2509.18063.

[15] Khorshidi, S. et al. (Apple Inc.). *ODKE+: Ontology-Driven Knowledge Extraction at Web Scale*. arXiv:2509.04696, 2025. https://arxiv.org/html/2509.04696v1.

[16] "Sentence segmentation for large language models," arXiv preprint arXiv:2601.03276, 2026. https://arxiv.org/html/2601.03276v1

[17] Neo4j Lexical Graph With Extracted Entities https://neo4j.com/blog/developer/graphrag-field-guide-rag-patterns/